

Sheriyans Cyber School

1: Why This VAPT Series Matters

1.1 Introduction

Vulnerability Assessment and Penetration Testing (VAPT) is often misunderstood as hacking or breaking systems just for fun.

In reality, VAPT is a **professional and structured cybersecurity practice** used by organizations to protect their systems and data.

This chapter explains:

- what VAPT really is
- what beginners usually misunderstand
- what this series aims to teach
- how VAPT knowledge is used in real-world jobs

Understanding this chapter is important because it builds the **foundation and mindset** required for learning VAPT correctly.

1.2 What VAPT Actually Is

VAPT stands for **Vulnerability Assessment and Penetration Testing**.

It is **not random hacking** and **not illegal activity**.

VAPT is a **systematic security process** in which security professionals:

- identify weaknesses in systems, networks, or applications
 - check whether those weaknesses can actually be exploited
 - evaluate how serious the risk is
 - help organizations fix security issues before attackers exploit them
-

1.2.1 Purpose of VAPT in the Real World

In real organizations, VAPT is performed to:

- understand how a real attacker might think and behave
- discover security weaknesses **before criminals do**
- reduce risks related to business, data, and reputation
- strengthen overall security posture

VAPT helps organizations stay **proactive**, not reactive.

1.3 Common Misunderstandings Among Beginners

Many beginners have a wrong idea about what VAPT actually means.

They believe VAPT is mainly about:

- running hacking tools
- exploiting every vulnerability found
- getting shells, access, or screenshots

This mindset is **incorrect and risky**.

1.3.1 What VAPT Is Really About

In professional security testing, VAPT focuses on:

- clearly knowing **what should be tested and what should not be tested**
- understanding **why a particular vulnerability is important**
- measuring the real impact of a vulnerability
- stopping the test at the **right time** to avoid damage

A good VAPT professional knows:

- how to attack
- when to stop
- how to stay within ethical and legal boundaries

This series corrects beginner misunderstandings and builds the **right security mindset from day one**.

1.4 Scope of This VAPT Series

This series is designed with a **clear learning scope**.

1.4.1 What This Series WILL Cover

This series will cover:

- real-world VAPT methodology used in industry
- how security professionals approach testing
- practical and responsible testing techniques
- attacker-style thinking with control and ethics

The focus is on **understanding concepts**, not blindly following tools.

1.4.2 What This Series WILL NOT Cover

This series will NOT include:

- illegal or unauthorized hacking
- irresponsible exploitation techniques
- shortcut-based learning approaches
- tool-only demonstrations without proper explanation

The objective is **long-term skill development**, not quick results.

1.5 Use of VAPT Knowledge in Real Jobs

In real cybersecurity roles, VAPT knowledge is applied to:

- test web applications, networks, and infrastructure
 - check whether security controls are working properly
 - support compliance requirements and security audits
 - communicate security risks to both technical and non-technical teams
-

1.5.1 What Really Matters in Security Jobs

In professional roles, what matters most is **not**:

- how many tools you know
- how many exploits you can run

What truly matters is:

- how clearly you can **identify security risks**
- how effectively you can **prove vulnerabilities**
- how well you can **explain impact and risk** to others

Clear communication is a **key security skill**.

2: Why Cybersecurity Is Different from Other IT Fields

2.1 Introduction

Cybersecurity is fundamentally different from other IT fields because the **nature of problems in security is intentional, not accidental**.

In traditional IT roles, issues usually happen due to mistakes or technical failures.

In cybersecurity, problems exist because **someone is actively trying to attack the system**.

This chapter explains why cybersecurity requires:

- a different mindset
- a different approach
- and different thinking compared to normal IT work

This understanding is essential for learning **VAPT and professional security testing**.

2.2 Why Security Problems Are Not Accidental

2.2.1 Nature of Traditional IT Problems

Most IT problems occur because of unintentional mistakes, such as:

- servers going down
- wrong configurations
- failed updates
- human errors

These problems are:

- accidental
 - technical
 - usually fixed once and do not repeat in the same way
-

2.2.2 Nature of Cybersecurity Problems

Cybersecurity problems are **intentional by nature**.

In security incidents:

- an attacker is **actively trying to break, bypass, or misuse a system**
- the attacker **thinks and plans attacks**
- the attacker **adapts when defenses change**
- the attacker **tries repeatedly until successful**

This means security threats are:

- intelligent
- persistent
- adaptive

Security is not about fixing errors;

it is about **defending against intelligent opposition**.

That is why cybersecurity always involves a **human mindset**, not just tools or technology.

2.3 Bugs vs Vulnerabilities

Understanding the difference between **bugs** and **vulnerabilities** is extremely important in cybersecurity and especially in **VAPT**.

2.3.1 What Is a Bug?

In software development, a **bug** is usually:

- a programming mistake
- an error in logic
- an issue that causes the application to behave incorrectly

Examples of bugs include:

- application crashes
- wrong output
- features not working as expected

👉 Bugs mainly affect **functionality**, not security.

2.3.2 What Is a Vulnerability?

A **vulnerability** is a **security weakness** in a system, application, network, or process that can be **intentionally exploited by an attacker**.

Unlike normal bugs, vulnerabilities are dangerous because they have **direct security impact**.

2.3.2.1 Intentional Exploitation

A vulnerability is a weakness that:

- attackers can **identify deliberately**
- can be abused in a planned way
- can be exploited again and again

Attackers actively search for vulnerabilities because these weaknesses can be turned into **attack entry points**.

2.3.2.2 Unauthorized Access or Actions

When a vulnerability is exploited, it may allow an attacker to:

- access sensitive data without permission
- bypass authentication or authorization controls
- perform actions as another user or administrator

This results in a **loss of control** over the system.

2.3.2.3 Security Impact of Vulnerabilities

Exploiting vulnerabilities can cause serious damage, such as:

a) Data Leakage

- exposure of sensitive user or business data
- violation of privacy and legal requirements

b) Account Takeover

- attacker gains control of user or admin accounts
- misuse of identity and privileges

c) System Compromise

- full control over servers or applications
- installation of malware or backdoors

These impacts affect:

- confidentiality
 - integrity
 - availability
-

2.3.2.4 Why Vulnerabilities Are More Serious Than Bugs

- Bugs affect how a system works
- Vulnerabilities affect **how secure a system is**

Even if an application works perfectly,
a single vulnerability can make the entire system **unsafe**.

👉 That is why **vulnerabilities affect security, not just functionality**.

2.3.3 Importance of Bugs vs Vulnerabilities in VAPT

In VAPT:

- professionals focus mainly on vulnerabilities
- attackers exploit vulnerabilities, not random bugs
- vulnerabilities represent **real-world risk**

Understanding this difference helps testers:

- prioritize findings
 - report meaningful risks
 - avoid false security assumptions
-

2.4 Why Traditional IT Troubleshooting Fails in Cybersecurity

2.4.1 Traditional Troubleshooting Approach

In normal IT work, troubleshooting usually follows this process:

1. something breaks
2. the issue becomes visible
3. investigation is done
4. the problem is fixed

This approach works well for operational issues.

2.4.2 Why This Approach Is Not Enough for Security

In cybersecurity:

- systems may appear to work normally

- there may be no error messages
- users may not notice anything wrong

Yet:

- attackers may already have access
- data may already be exposed
- hidden weaknesses may still exist

Security testing requires:

- assuming something is wrong
- questioning normal system behavior
- looking for hidden weaknesses
- thinking from an attacker's perspective

Because of this, a troubleshooting mindset alone is **not sufficient**.

Cybersecurity requires **adversarial thinking**.

2.5 Adversarial Thinking in Cybersecurity

Adversarial thinking means thinking like an attacker while acting responsibly.

It involves:

- asking "How can this be misused?"
- identifying abuse cases
- predicting attacker behavior

This mindset is essential for:

- VAPT
 - penetration testing
 - red teaming
 - defensive security roles
-

2.6 Key Takeaway

In IT, you fix what is broken.

In cybersecurity, you **hunt what is hiding**.

This statement clearly captures the **core difference** between IT and cybersecurity and is useful for exams, interviews, and presentations.

3: Categories of Penetration Testing

3.1 Introduction

When an organization plans a penetration test, defining the **scope** is not the only important step.

Another critical decision is **how much information will be shared with the penetration tester**.

The amount of information provided to the tester directly affects:

- how the test is performed
- what kind of attacker is being simulated
- the depth and realism of the testing

Based on this, penetration testing is commonly divided into **three main categories**:

1. Black Box Penetration Testing
2. White Box Penetration Testing
3. Gray Box Penetration Testing

Each category serves a **different purpose** and represents a **different attack scenario**.

3.2 Black Box Penetration Testing

3.2.1 Definition

In **Black Box Penetration Testing**, the tester is given **little or no internal information** about the target system.

The tester approaches the system in the same way an **external attacker** would, without any insider knowledge.

3.2.2 Information Provided to the Tester

In a Black Box test, the tester usually has:

- no knowledge of internal architecture
- no access to source code
- no details about system configuration or versions

The tester only knows:

- the target domain, IP, or application
 - what is publicly accessible
-

3.2.3 Objective of Black Box Testing

The main objectives of Black Box testing are to:

- test **external attack surface**
- identify publicly exposed weaknesses
- evaluate real-world attack paths
- understand what an outsider can discover and exploit

This type of testing answers the question:

“What can an attacker find and exploit from the outside?”

3.2.4 Skills Required in Black Box Testing

Black Box testing heavily depends on:

- reconnaissance skills

- information gathering techniques
- enumeration and discovery
- attacker-style thinking

Because the tester starts with almost no information, **discovery is a major part of the test.**

3.2.5 Use Cases of Black Box Testing

Black Box testing is commonly used when:

- testing internet-facing systems
- simulating real-world external attacks
- assessing perimeter security

It provides a **realistic attacker simulation**, but may not cover all internal issues.

3.3 White Box Penetration Testing

3.3.1 Definition

In **White Box Penetration Testing**, the organization shares **almost complete internal information** with the tester.

This approach focuses on **depth and completeness**, not attacker realism.

3.3.2 Information Provided to the Tester

White Box testing may include access to:

- system and network architecture
- application design details
- service versions and configurations
- source code (especially for applications)

The tester has **maximum visibility** into the system.

3.3.3 Objective of White Box Testing

The goal of White Box testing is to:

- deeply analyze security weaknesses
- identify flaws in design and implementation
- validate security controls at code level
- ensure secure development practices

It answers the question:

“Are there any security weaknesses inside the system?”

3.3.4 Advantages of White Box Testing

White Box testing allows:

- thorough security coverage
- faster identification of critical vulnerabilities
- detailed and accurate findings
- early detection of security flaws

This makes it very effective for:

- internal security testing
 - secure software development lifecycle (SDLC)
 - compliance-driven assessments
-

3.3.5 Limitations of White Box Testing

Although very detailed, White Box testing:

- does not simulate a real external attacker
 - may miss issues related to real-world discovery
 - assumes insider-level access
-

3.4 Gray Box Penetration Testing

3.4.1 Definition

Gray Box Penetration Testing is a **balanced approach** between Black Box and White Box testing.

The tester is given **limited but useful information**, while some details are intentionally withheld.

3.4.2 Information Provided to the Tester

In Gray Box testing, the tester may receive:

- application names
- test or low-privilege user accounts
- partial knowledge of the system

However, the tester does NOT receive:

- full architecture details
 - complete configuration information
 - exact service versions
-

3.4.3 Objective of Gray Box Testing

The objectives of Gray Box testing are to:

- simulate a **semi-trusted user**
- test realistic attack scenarios
- achieve good coverage without full disclosure
- identify both external and internal weaknesses

It answers the question:

“What can a user with limited access do if they try to attack the system?”

3.4.4 Why Organizations Prefer Gray Box Testing

Many organizations prefer Gray Box testing because:

- it balances realism and efficiency
- it provides better coverage than Black Box
- it is more realistic than White Box
- it saves time and cost

Gray Box testing often reflects **real-world insider threat scenarios**.

3.5 Comparison of Black Box, White Box, and Gray Box Testing

Aspect	Black Box	White Box	Gray Box
Information Shared	None / Very Limited	Almost Complete	Partial
Attacker Simulation	External Attacker	Insider / Developer	Semi-Trusted User
Coverage	Limited	High	Medium to High
Realism	High	Low	Balanced
Time Required	High	Low	Medium

4: Important Cybersecurity Terminologies

4.1 Introduction

In cybersecurity and VAPT, understanding terminology is **not optional**.

These terms are used daily in:

- penetration testing reports
- risk discussions
- audits and compliance
- security meetings with management

Without clear understanding of these terms, a tester may:

- misunderstand risks
- miscommunicate findings
- fail to justify security impact

This chapter explains **core cybersecurity terms** in a **simple but professional way**.

4.2 Core Cybersecurity Terminologies

4.2.1 Asset

An **asset** is **anything that has value and must be protected**.

In cybersecurity, assets are not limited to physical systems like servers.

Assets can include:

- sensitive data (user data, credentials, financial records)
- applications and software
- servers, networks, and cloud infrastructure
- services that support business operations

An asset is defined by its **value to the organization**, not by its technical nature.

Security Mindset

Before starting any security test, the first question should always be:

| What are we protecting?

Without identifying assets, security testing has **no clear direction**.

4.2.2 Vulnerability

A **vulnerability** is a **weakness or flaw** present within an asset.

This weakness may be:

- technical (software bugs, outdated versions)
- logical (broken authentication or authorization)

- configuration-related (misconfigurations, weak permissions)

A vulnerability **does not mean the system is already hacked**.

It only means that **there is a possibility of compromise**.

Important Distinction

- Vulnerability represents **potential**
- It does NOT represent successful exploitation

Understanding this prevents panic and helps in **proper risk evaluation**.

4.2.3 Threat

A **threat** represents a **potential danger** to an asset.

Anything that has the **intent and capability** to exploit a vulnerability is considered a threat.

Threats can be:

- human attackers
- malicious insiders
- malware or automated scripts
- natural or environmental factors

Threats can exist even if no vulnerability is known.

Core Concept

Threat is about:

- **intent** (willingness to attack)
- **capability** (ability to attack)

Not about execution.

4.2.4 Exploit

An **exploit** is a **method, technique, or code** used to take advantage of a vulnerability.

Exploits convert:

- a theoretical weakness
into
- an actual compromise

An exploit acts as a **bridge** between:

- vulnerability
- unauthorized access or impact

Professional VAPT Mindset

The goal is NOT:

- running exploits blindly

The real goal is:

- proving the **impact** safely
 - demonstrating **real risk**
-

4.2.5 Risk

Risk refers to the **potential damage or loss** resulting from a successful exploitation.

Risk is NOT decided by vulnerability alone.

Risk depends on:

- likelihood of exploitation
- impact on the organization
- value of the affected asset

Simplified Formula (Conceptual)

| Risk = Likelihood × Impact

Simple Understanding

Risk answers the question:

| How bad can things get if this vulnerability is exploited?

This is the most important question for **business decision-making**.

4.3 Additional Core Terminologies (Must-Know)

4.3.1 Attack Surface

The **attack surface** is the **total set of all possible entry points** where an attacker can try to gain access.

This includes:

- open ports
- exposed services
- APIs
- user input fields
- public-facing applications

A **larger attack surface** means:

- more attack opportunities
- higher security risk

Reducing attack surface is a key security goal.

4.3.2 Attack Vector

An **attack vector** is the **specific path or technique** used to perform an attack.

Difference between the two:

- Attack surface = *where* an attack is possible
- Attack vector = *how* the attack is carried out

Example:

- login page = attack surface

- SQL injection = attack vector

Understanding attack vectors helps testers **anticipate attacker behavior**.

4.3.3 Proof of Concept (PoC)

A **Proof of Concept (PoC)** demonstrates that a vulnerability is:

- real
- exploitable
- not just theoretical

A PoC removes doubt and provides **clear evidence**.

Without a PoC:

- findings may be questioned
- vulnerabilities may be ignored

PoC is essential for **credible VAPT reporting**.

4.3.4 Impact

Impact describes the **consequences** of a successful exploit.

It explains:

- what an attacker can achieve
- why the issue matters
- how the business will be affected

Impact can be:

- technical (system compromise)
- financial (loss of revenue)
- legal (compliance violations)
- reputational (loss of trust)

Impact connects **technical issues to business risk**.

4.3.5 False Positive

A **false positive** occurs when a vulnerability is reported but **does not actually exist**.

Automated security tools often generate false positives.

That is why:

- human validation is mandatory
- manual testing is important

Reporting false positives can:

- waste time
- reduce trust in security teams
- create unnecessary panic

Professional testers always **verify before reporting**.

4.4 Relationship Between Key Terms (Exam Important)

- Assets are what we protect
- Vulnerabilities are weaknesses in assets
- Threats are potential attackers
- Exploits are methods used to attack
- Risk is the possible damage
- Impact explains why it matters

Understanding these relationships is **critical for VAPT**.

5: VAPT Methodology Overview

5.1 Introduction

Professional VAPT is **never performed randomly**.

Security experts do not jump directly into tools or exploitation.

Instead, they follow a **structured, step-by-step methodology** to ensure that testing is:

- controlled
- repeatable
- reliable
- explainable to stakeholders

The main goal of VAPT is **not to break systems**.

The real goal is to **identify, validate, and clearly communicate security risk**.

Every phase in the methodology has a **specific purpose**.

If any step is skipped, the **quality and credibility of the final results decrease**.

5.2 How Professionals Approach VAPT

Professional VAPT follows a **disciplined and logical approach**.

Security professionals focus on:

- understanding the system before attacking
- following defined scope and authorization
- validating findings instead of guessing
- minimizing risk to production systems

5.2.1 Purpose of a Structured Methodology

A structured methodology ensures that:

- testing can be repeated by other testers
- results can be verified and trusted
- findings can be explained to management
- legal and ethical boundaries are respected

Without a methodology, VAPT becomes:

- noisy
 - unreliable
 - difficult to defend in reports
-

5.2.2 Core Objective of VAPT

The objective of VAPT is **not destruction**.

The real objectives are to:

- identify security weaknesses
- validate whether they are exploitable
- measure business and technical impact
- communicate risk clearly to decision-makers

👉 VAPT exists to **support risk management**, not to show hacking skills.

5.3 Reconnaissance Phase in VAPT

5.3.1 What Is Reconnaissance?

Reconnaissance (Recon) is the **foundation phase** of VAPT.

In this phase, the tester focuses on **understanding the target**, not attacking it.

Recon helps answer questions like:

- What systems exist?
 - What services are exposed?
 - What technologies are being used?
-

5.3.2 Purpose of Reconnaissance

During reconnaissance:

- the system is carefully studied
 - the **attack surface** is identified
-

- potential weak areas are shortlisted
- unnecessary noise is avoided

Good reconnaissance is:

- silent
- systematic
- focused

👉 The stronger the reconnaissance, the more **focused and effective** the later stages become.

5.3.3 Common Beginner Mistake

Most beginners make a serious mistake during recon.

They:

- rush through information gathering
- ignore understanding the system
- jump directly to exploitation

This leads to:

- missed vulnerabilities
- false positives
- noisy and unprofessional testing

Professional VAPT always respects **recon as a critical phase**.

5.4 Exploitation Phase in VAPT

5.4.1 When Exploitation Is Performed

Exploitation is performed **only after**:

- a vulnerability is clearly identified
- the tester understands the potential impact

- scope and authorization are confirmed

Exploitation is **not the starting point** of VAPT.

5.4.2 Responsible Use of Exploitation

Professionals do NOT exploit every vulnerability they find.

Exploitation is performed only when:

- proving the risk is necessary
- business impact must be demonstrated
- assumptions need to be converted into facts

Unnecessary exploitation can:

- damage systems
 - cause downtime
 - violate scope
-

5.4.3 Role of Exploitation in VAPT

The purpose of exploitation is to:

- confirm that a vulnerability is real
- demonstrate possible attacker actions
- validate severity and impact

👉 Exploitation converts:

- **assumptions → evidence**
- **theory → reality**

This makes findings **credible and actionable**.

5.5 Importance of Reporting in VAPT

5.5.1 Reporting as the Final Deliverable

The **most important output** of VAPT is not shells or screenshots.

It is the **final report**.

No matter:

- how deep the vulnerability is
- how complex the exploit was

If the report is unclear, the **value of the entire VAPT becomes zero**.

5.5.2 Why Reporting Matters More Than Hacking

Organizations do not pay for hacking skills.

They pay for:

- clarity
- understanding
- decision support

A good VAPT report clearly explains:

- what the issue is
 - why it is dangerous
 - how serious the risk is
 - how it can be fixed
-

5.5.3 Audience of a VAPT Report

A VAPT report is read by:

- technical teams (developers, admins)
- security teams
- management and executives

Therefore, the report must:

- be technically accurate
- be easy to understand

- clearly communicate risk and impact
-

5.6 Relationship Between VAPT Phases

- Reconnaissance builds understanding
- Exploitation validates risk
- Reporting communicates value

Skipping any phase results in:

- incomplete testing
 - weak findings
 - poor decision-making
-

6: Types of Penetration Testing

6.1 Introduction

Penetration testing is not limited to only one type of system or environment.

In real-world engagements, organizations perform **different types of penetration tests** depending on:

- the nature of their infrastructure
- the kind of data they handle
- their threat model
- compliance and business requirements

Each type of penetration test focuses on **different assets, attack surfaces, and threat scenarios**.

This chapter explains the **most commonly performed penetration tests** in professional environments.

6.2 Network Penetration Testing

6.2.1 Overview

Network Penetration Testing focuses on identifying security weaknesses in a network environment.

It evaluates:

- network architecture
- exposed services
- firewall and access control rules
- segmentation and trust boundaries

Network testing helps organizations understand how attackers can move through their network.

6.2.2 External Network Penetration Testing

Definition

An **External Network Penetration Test** targets systems that are **publicly accessible from the internet**.

It simulates an attacker who is **outside the organization**.

Key Characteristics

- targets public IP addresses and domains
 - no internal network access
 - attacker has no insider privileges
-

Objectives

External network testing is performed to:

- test firewall and perimeter security
- identify exposed or misconfigured services
- evaluate real-world external attack paths

This type of test answers the question:

| “What can an internet-based attacker do to our organization?”

6.2.3 Internal Network Penetration Testing

Definition

An **Internal Network Penetration Test** is performed **from inside the organization’s network**.

It simulates:

- a malicious insider
 - or an attacker who has already gained internal access
-

Access Methods

Internal access may be provided through:

- VPN credentials
 - a dedicated test system
 - physical presence inside the organization
-

Objectives

Internal testing focuses on:

- lateral movement within the network
- privilege escalation
- access to sensitive systems and data
- weaknesses in internal trust relationships

This test answers:

| “What damage can be done after internal access is obtained?”

6.3 Web Application Penetration Testing

6.3.1 Overview

Web Application Penetration Testing is one of the **most commonly performed tests today**.

This is because web applications often handle highly sensitive data, such as:

- usernames and passwords
 - personal and financial information
 - payment and transaction data
-

6.3.2 Focus Areas

Web application testing focuses on identifying vulnerabilities related to:

- authentication (login mechanisms)
- authorization (access control)
- session management
- input validation
- business logic

These vulnerabilities often cannot be detected by network testing alone.

6.3.3 Importance

A single vulnerability in a web application can lead to:

- data breaches
- account takeovers
- regulatory violations

That is why web application penetration testing is **critical for modern organizations**.

6.4 Mobile Application Penetration Testing

6.4.1 Overview

With the widespread use of smartphones, **mobile application penetration testing** has become increasingly important.

Organizations rely heavily on:

- Android applications
- iOS applications

to deliver services to users.

6.4.2 Objectives

Mobile penetration testing aims to ensure that mobile applications:

- securely store user data
 - communicate securely with backend servers
 - do not expose sensitive information through logs or local storage
 - properly authenticate and authorize users
-

6.4.3 Common Risk Areas

Mobile testing focuses on:

- insecure local storage
- weak API security
- improper encryption
- insecure communication channels

Mobile apps often act as **gateways to backend systems**, making their security critical.

6.5 Social Engineering Penetration Testing

6.5.1 Overview

Social Engineering Penetration Testing focuses on the **human element of security**.

Instead of attacking systems, the tester targets **people**.

6.5.2 Common Techniques

Social engineering tests may include:

- phishing emails
- spear-phishing campaigns
- browser-based attacks
- credential harvesting attempts

These techniques attempt to **trick users** into performing actions they did not intend to perform.

6.5.3 Objectives

The objective of social engineering testing is to:

- measure employee security awareness
- evaluate the effectiveness of training programs
- identify weaknesses in human behavior

It answers the question:

| "Can attackers manipulate our employees to bypass security?"

6.6 Physical Penetration Testing

6.6.1 Overview

Physical Penetration Testing focuses on **physical security controls**.

Although less common, it is important in environments where physical access can lead to serious compromise.

6.6.2 Scope of Physical Testing

Physical testing may involve attempting to:

- access restricted areas
 - bypass locks or access cards
 - test RFID or biometric systems
 - evaluate surveillance and monitoring
-

6.6.3 Importance

Physical access can allow attackers to:

- steal devices
- connect rogue hardware
- access internal networks

This makes physical security an important part of **overall organizational security**.

6.7 Report Writing in Penetration Testing

6.7.1 Importance of Reporting

The **penetration test report** is the **most critical deliverable** of any engagement.

Even a technically successful test becomes **useless** if the report is unclear or poorly written.

Organizations do not pay for testing alone;

they pay for **clear understanding and actionable results**.

6.7.2 Key Characteristics of a Good Penetration Test Report

A professional penetration testing report should:

- be simple, clear, and easy to understand
- follow consistent formatting and structure
- use correct spelling and grammar
- maintain a professional writing tone
- eliminate false positives through validation

- analyze each vulnerability in detail
 - explain root causes, not just symptoms
 - include evidence such as screenshots, logs, or requests
-

6.7.3 Purpose of Evidence in Reports

Evidence helps to:

- prove that a vulnerability is real
- remove ambiguity
- build trust with stakeholders

Good evidence makes findings **credible and defensible**.

7: Understanding the Audience of a Penetration Testing Report

7.1 Introduction

A penetration testing report is **not written for only one type of reader**.

In real-world VAPT engagements, the same report is read by **multiple stakeholders**, and each of them has:

- different responsibilities
- different technical knowledge
- different expectations

If the report does not match the audience, even **high-quality testing loses its value**.

That is why understanding the **audience of a VAPT report** is a critical professional skill.

7.2 Why Audience Understanding Is Important

Different readers look for **different answers** in the report.

- Executives want to know **risk and business impact**
- Managers want to know **priorities and decisions**
- Technical teams want to know **exact technical details and fixes**

A single report must communicate **effectively with all of them**.

A good VAPT report is not overly technical,

and it is not overly vague either —

it is **well-structured**.

7.3 Executive Class

7.3.1 Who Belongs to This Group

The **Executive Class** includes:

- CEOs
- founders
- board members
- top-level leadership

These individuals are usually **non-technical** and have **very limited time**.

7.3.2 What Executives Look For

Executives do NOT read the full technical report.

They usually focus only on:

- the **Executive Summary**
- overall security risk level
- business impact
- high-level recommendations

They want answers to questions like:

- *Is the organization at risk?*
 - *How serious is the problem?*
 - *Does this affect business, reputation, or compliance?*
-

7.3.3 How Information Should Be Presented

For executives, the report should:

- avoid technical jargon
- use clear and simple language
- focus on **risk, impact, and consequences**

Executives care about **decisions**, not technical details.

7.4 Management Class

7.4.1 Who Belongs to This Group

The **Management Class** includes:

- CISOs
- security managers
- IT managers
- compliance and risk officers

They usually have **some technical understanding**, but their role is **strategic**, not hands-on.

7.4.2 What Management Looks For

Management focuses on:

- overall security posture
- strengths and weaknesses
- severity and risk levels
- remediation priorities

They want to understand:

- *Which issues are most dangerous?*
 - *What should be fixed first?*
 - *How should resources be allocated?*
-

7.4.3 Role of the Report for Management

For management, the report acts as a:

- decision-making document
- risk management tool
- planning reference

Clear prioritization helps management **take action efficiently**.

7.5 Technical Class

7.5.1 Who Belongs to This Group

The **Technical Class** includes:

- developers
- system administrators
- security engineers
- DevOps teams

These readers are **highly technical** and work directly on systems.

7.5.2 What Technical Teams Look For

Technical teams read the report in **full detail** to:

- understand how vulnerabilities work
- review proof of concept (PoC)
- analyze root causes

- apply fixes and patches

They need:

- exact vulnerability details
 - reproduction steps
 - affected components
 - remediation guidance
-

7.5.3 Importance of Accuracy for Technical Audience

For technical readers:

- vague explanations are useless
- false positives reduce trust
- missing details slow down fixes

A good report helps technical teams **fix issues correctly and permanently**.

7.6 Structuring a Report for Multiple Audiences

A professional VAPT report is structured so that:

- executives understand **risk and impact**
- management understands **priorities and actions**
- technical teams understand **how to fix vulnerabilities**

This is usually achieved by:

- executive summary at the beginning
- risk overview and prioritization
- detailed technical findings later in the report

Each section serves a **different audience**, but all are connected.

7.7 Common Mistakes When Audience Is Ignored

Ignoring the audience can lead to:

- executives not reading the report
- management not taking action
- technical teams misunderstanding issues

This results in:

- wasted testing effort
 - unresolved risks
 - poor security outcomes
-

Chapter 8: Reconnaissance (Recon) in VAPT

8.1 Introduction to Reconnaissance

Reconnaissance (Recon) is the **first and one of the most critical phases of VAPT**.

In this phase, the attacker or penetration tester focuses on **information gathering**, not exploitation.

The goal of recon is **not to hack the system**,

but to **understand the target as deeply as possible** before any attack is attempted.

Recon answers one fundamental question:

“What do I know about the target before attacking it?”

A strong recon phase leads to:

- focused testing
- fewer mistakes
- less noise

- higher success rate

Skipping or rushing recon always results in **weak exploitation and poor findings**.

8.2 Role of Reconnaissance in VAPT Methodology

Reconnaissance acts as the **foundation** of the entire VAPT process.

During recon:

- the target environment is studied
- the attack surface is identified
- assumptions are formed for later validation

Recon does not directly cause damage,
but it **decides the quality of everything that comes after it**.

8.3 Types of Reconnaissance

Reconnaissance is broadly divided into **two main types**:

1. Passive Reconnaissance
2. Active Reconnaissance

Each type serves a **different purpose** and has a **different risk level**.

8.4 Passive Reconnaissance

8.4.1 Definition

Passive Reconnaissance means collecting information **without directly interacting with the target systems**.

Key characteristics:

- no packets sent to target infrastructure
- no direct probing
- very low chance of detection

The tester behaves like a **silent observer**, not an attacker.

8.4.2 Passive Recon Mindset

You are observing from a distance,
not touching the target.

The goal is to learn **as much as possible without creating logs or alerts**.

This approach is heavily used in:

- early-stage penetration testing
 - red team operations
 - intelligence gathering
-

8.4.3 Information Gathered During Passive Recon

Passive reconnaissance helps collect information such as:

- domains and subdomains
- indirect IP range information
- technology stack (frameworks, CMS, cloud providers)
- employee names, emails, usernames
- public documents and metadata
- exposed credentials or code references
- DNS records (without aggressive querying)

All this information is obtained from **publicly available sources**.

8.4.4 Why Passive Recon Is Important

Passive reconnaissance:

- builds a **mental map of the organization**
- reduces noise before active scanning
- lowers the risk of detection

- forms the base for attack surface discovery

Although passive recon is low risk and silent,
it provides **high-value intelligence** when done properly.

8.5 Active Reconnaissance

8.5.1 Definition

Active Reconnaissance involves **direct interaction with the target systems** to gather technical details.

Unlike passive recon, here the tester **touches the target**, which means:

- logs may be generated
 - alerts may be triggered
-

8.5.2 Active Recon Mindset

You are now confirming assumptions
made during passive reconnaissance.

Active recon is about **verification**, not guessing.

8.5.3 Information Gathered During Active Recon

Active reconnaissance helps identify:

- live hosts and reachable systems
- open ports
- running services and their versions
- operating system details
- directories and files
- API endpoints
- application behavior

- network responses and service banners

This information is **technical, precise, and actionable**.

8.5.4 Why Active Recon Is Important

Active reconnaissance:

- converts theory into **verified data**
- identifies **actual attack vectors**
- is required before exploitation
- supports accurate vulnerability validation

Without active recon, exploitation becomes **blind and unreliable**.

8.5.5 Risk Level of Active Recon

The risk level of active reconnaissance is:

- **medium to high**, depending on
 - intensity of scanning
 - tools used
 - scope and timing

That is why active recon must always respect:

- scope
 - authorization
 - rate limits
-

8.6 Passive vs Active Reconnaissance (Core Difference)

Aspect	Passive Recon	Active Recon
Interaction with Target	No	Yes
Detectability	Very low	Possible

Aspect	Passive Recon	Active Recon
Goal	Information discovery	Validation & confirmation
Noise Level	Minimal	Can be noisy
Used In	Early recon, red teaming	Pentesting, exploit prep

Both are essential and **complement each other**.

8.7 Footprinting in Reconnaissance

8.7.1 What Is Footprinting?

Footprinting is a **pre-recon activity** performed before deep scanning.

The objective of footprinting is to:

- understand the target at a **high level**
- collect the **outer layer of information**
- avoid blind or out-of-scope testing

Think of footprinting as **understanding the shape of the target**, not attacking it.

8.8 Categories of Information Obtained During Footprinting

Information collected during footprinting can be divided into **three major categories**.

8.8.1 Organization-Level Information

This focuses on **understanding the organization itself**, not its systems yet.

What Is Collected

- organization name, brand, subsidiaries
- business model and industry
- office locations and global presence

- email formats and naming conventions
 - public contacts and departments
 - technology clues from job postings
 - third-party services and vendors
-

Why Organization-Level Info Matters

This information:

- helps in **target profiling**
- indirectly reveals attack surface
- provides context for technical findings
- is critical for understanding human risk

It answers:

| Who is the target and why it exists?

8.8.2 Network-Level Information

This focuses on **how the organization exists on the internet**.

What Is Collected

- public IP ranges
 - domains and subdomains
 - DNS records (A, MX, NS, TXT, etc.)
 - ASN and network block details
 - cloud service usage (AWS, Azure, GCP)
 - VPN gateways and exposed services
 - third-party hosted assets
-

Why Network-Level Info Matters

Network-level information:

- defines the **technical boundary**
- helps avoid out-of-scope testing
- forms the base for scanning and enumeration
- identifies externally exposed infrastructure

It answers:

| Where does the target live on the internet?

8.8.3 System-Level Information

This focuses on **individual systems and applications**, still at a high level.

What Is Collected

- operating systems (Windows, Linux, etc.)
 - web servers and application servers
 - frameworks and CMS platforms
 - database technologies
 - publicly visible software versions
 - security headers and configurations
 - application entry points
-

Why System-Level Info Matters

System-level information:

- helps identify **potential vulnerability classes**
- guides tool and technique selection
- reduces blind exploitation attempts
- makes testing structured and logical

It answers:

| What technologies are actually running?

9: Browser-Based Information Gathering

9.1 Introduction

Browser-based information gathering is an important part of **reconnaissance**, especially **passive recon**.

In this phase, the penetration tester uses:

- search engines
- publicly available data
- browser-accessible resources

to collect useful information **without directly attacking the target**.

One of the most powerful techniques in browser-based recon is **Google Dorking**.

9.2 Google Dorking

9.2.1 What Is Google Dorking? (Conceptual Understanding)

Normal users search on Google like this:

| "company login page"

A penetration tester searches like this:

| "Show me what the website owner did not intend to make public."

Google Dorking means:

| Using advanced Google search operators to find
sensitive, misconfigured, or accidentally exposed information.

9.2.2 Important Clarification (Ethics & Legality)

- Google itself does **not hack anything**
- The website owner **accidentally exposed** the data
- Google only **indexes and displays** what is already public

That is why Google Dorking is considered:

- **100% passive reconnaissance**
- legal when done within scope
- commonly used in professional VAPT

9.2.3 Basic Logic Behind Google Dorks

The logic of every Google dork is simple:

| "Only show results that match this condition."

These conditions may relate to:

- page title
- URL structure
- file type
- page content

You are basically **applying filters to Google search results**.

9.3 Common Google Search Operators (With Purpose)

| ⚠ Syntax is shown only for understanding, not misuse.

9.3.1 site: Operator

```
site:example.com
```


Purpose:

- limits results to a specific domain
- removes unrelated noise

Used to focus only on **one organization**.

9.3.2 inurl: Operator

```
inurl:admin
```

Purpose:

- searches for sensitive keywords in URLs
 - useful for finding admin, login, panel pages
-

9.3.3 intitle: Operator

```
intitle:login
```

Purpose:

- identifies page purpose using the title
 - login, dashboard, portal pages are often revealed
-

9.3.4 filetype: Operator

```
filetype:pdf
```

Purpose:

- finds non-HTML files
- documents, reports, manuals

Such files often contain **sensitive metadata**.

9.3.5 ext: Operator

```
ext:sql
```

Purpose:

- searches for specific file extensions
 - useful for finding backups and database dumps
-

9.3.6 intext: Operator

```
intext:password
```

Purpose:

- searches inside page content
 - helps identify sensitive keywords
-

9.3.7 cache: Operator

```
cache:example.com
```

Purpose:

- views Google's cached version of a page
 - useful when content is deleted or access-restricted
-

9.3.8 related: Operator

```
related:example.com
```

Purpose:

- identifies similar or related websites
 - useful for discovering sister domains
-

9.3.9 allinurl: Operator

```
allinurl:adminlogin
```

Purpose:

- forces multiple keywords into the URL
 - highly targeted discovery
-

9.3.10 allintitle: Operator

```
allintitle:admin panel
```

Purpose:

- finds pages with very specific titles
 - reduces false results
-

9.3.11 before: Operator

```
before:2020
```

Purpose:

- finds older content
 - older pages often contain unpatched issues
-

9.3.12 after: Operator

```
after:2024
```

Purpose:

- finds recent deployments
 - new systems often have misconfigurations
-

9.3.13 Logical Operators (OR / AND / -)

```
loginOR signin  
adminANDlogin  
login -help
```

Purpose:

- broaden or narrow searches
 - remove irrelevant results
-

9.3.14 Wildcard Operator (*)

```
"index of *"
```

Purpose:

- matches unknown words or paths
 - useful for directory listings
-

9.3.15 Exact Match Operator (" ")

```
"index of /backup"
```

Purpose:

- matches exact phrases
 - highly accurate results
-

9.3.16 inanchor: Operator

```
inanchor:login
```

Purpose:

- searches hyperlink text
 - helps identify hidden or unlinked pages
-

9.3.17 allintext: Operator

```
allintext:username password
```

Purpose:

- forces multiple sensitive words in content
 - useful for credential leakage discovery
-

9.3.18 Country-Specific Targeting

```
site:.in
```

Purpose:

- targets country-specific domains
 - useful for regional assessments
-

9.4 Combination Google Dorks (Real-World Usage)

Admin Panel Discovery

```
site:example.com inurl:admin intitle:login
```

Open Directory with Backups

```
site:example.com intitle:"index of" ext:bak
```

Exposed Configuration Files

```
site:example.com filetype:conf OR filetype:cfg
```

Database Dumps

```
site:example.com ext:sql OR ext:db
```

Staging or Test Environments

```
site:example.com inurl:test OR inurl:staging
```

Credential Leakage

```
site:example.com intext:"username" intext:"password"
```

Error-Based Reconnaissance

```
site:example.com intext:"SQL error"OR intext:"warning"
```

Forgotten Subdomains

```
site:*.example.com intitle:login
```

Cloud Storage Exposure

```
site:s3.amazonaws.com example
```

Cached Sensitive Pages

```
cache:example.com inurl:admin
```

9.5 DNS Enumeration

9.5.1 What Is DNS?

The **Domain Name System (DNS)** is a distributed database that translates:

| domain names → IP addresses

DNS is one of the **most critical internet systems**, making it a valuable recon target.

9.5.2 Common DNS Record Types

- **NS** – Authoritative name servers
 - **A** – IPv4 address of a domain
 - **AAAA** – IPv6 address
 - **MX** – Mail servers
 - **PTR** – Reverse lookup records
 - **CNAME** – Alias records
 - **TXT** – Arbitrary text (verification, SPF, etc.)
-

9.5.3 DNS Recon Commands (Basic)

```
host example.com
```

```
host -t mx example.com
```

9.6 DNS Cache Snooping

9.6.1 What Is DNS Cache Snooping?

DNS cache snooping is an **information-gathering technique** used to check:

| Which domains were recently resolved by a DNS server.

If a domain is cached, it usually means:

- someone on that network accessed it recently
-

9.6.2 Why Attackers Use DNS Cache Snooping

Attackers use it to:

- understand user browsing behavior
- identify frequently used services
- plan targeted phishing or social engineering

It is used in **reconnaissance**, not exploitation.

9.6.3 Methods of DNS Cache Snooping

1. Non-Recursive Method (Preferred)

- DNS server is asked **not to fetch new data**
- If record exists → it was already cached
- More accurate and reliable

```
dig @<dns_server><domain> A +norecurse
```

2. Recursive Method (Not Recommended)

- DNS server fetches data from internet
- TTL is checked to guess caching

```
dig @<dns_server><domain> A +recurse
```

Results are **unreliable**.

9.6.4 Using nslookup (Basic)

```
nslookup  
server <dns_server>
```

```
set norecurse  
<domain>
```

9.6.5 DNS Record Commonly Checked

- **A record**
- because it directly maps domain → IP

9.7 tracert / traceroute (Brief Concept)

`tracert` (Windows) or `traceroute` (Linux) is used to:

- identify network path
- discover intermediate hops
- understand routing behavior

Useful for:

- network mapping
- identifying firewalls and gateways

10: Advanced Reconnaissance & Enumeration Techniques

10.1 Introduction

After basic reconnaissance and browser-based information gathering, a VAPT professional moves towards **advanced recon and enumeration**.

This stage focuses on:

- validating assumptions
- extracting technical details

- identifying real attack paths

Most techniques in this chapter fall under **active reconnaissance**, meaning:

- the target is directly interacted with
- logs may be generated
- scope and authorization are mandatory

This phase converts **information into actionable intelligence**.

10.2 Domain & Ownership Reconnaissance

10.2.1 WHOIS Lookup

WHOIS is used to identify **domain ownership and registration details**.

It can reveal:

- domain owner (sometimes masked)
- registration and expiry dates
- registrar information
- name servers

This helps understand:

- domain lifecycle
 - potential forgotten assets
 - external dependencies
-

10.2.2 host Command (Basic DNS Info)

The `host` command is used to resolve domain names and fetch DNS records.

```
host example.com
```

To query specific record types:

```
host -t mx example.com  
host -t ns example.com
```

This provides a **quick DNS overview** during recon.

10.3 Source Code & Repository Reconnaissance

Modern organizations often leak sensitive data through **public code repositories**.

10.3.1 GitRob (Large Repositories)

GitRob is used to scan **large GitHub organizations** for sensitive files.

It searches for:

- API keys
- credentials
- configuration files
- backup files

GitRob is useful when:

- organization has multiple repositories
 - manual review is impractical
-

10.3.2 GitLeaks (Secrets Detection)

GitLeaks focuses on detecting **hardcoded secrets** in repositories.

It can identify:

- passwords
- API tokens
- cloud keys
- database credentials

This type of leak often leads to **direct compromise**.

10.4 Internet-Wide Reconnaissance

10.4.1 Shodan

Shodan is a search engine for **internet-connected devices**.

It allows testers to find:

- exposed servers
- IoT devices
- webcams
- databases
- outdated services

Shodan helps answer:

“What does this organization expose to the internet?”

Used heavily in:

- external recon
 - attack surface discovery
-

10.4.2 Security Headers Check

Website:

<https://securityheaders.com/>

This tool checks:

- HTTP security headers
- missing or misconfigured protections

Examples:

- Content-Security-Policy
- X-Frame-Options

- HSTS

Missing headers do not hack systems directly,
but they **increase attack surface**.

10.5 Cryptographic & Protocol Weakness Checks

10.5.1 POODLE

POODLE is a vulnerability affecting **SSLv3**.

It allows:

- downgrade attacks
- encryption compromise

Discovery of POODLE indicates:

- outdated cryptographic configuration
-

10.5.2 Heartbleed

Heartbleed affects **OpenSSL**.

It allows attackers to:

- read server memory
- extract sensitive data like keys and passwords

Finding Heartbleed is **critical severity**.

10.6 DNS Reconnaissance & Enumeration

10.6.1 dnsrecon

`dnsrecon` automates DNS enumeration.

Standard scan:

```
dnsrecon -d example.com -t std
```

Brute-force subdomain enumeration:

```
dnsrecon-dexample.com-Dsubdomains.txt-tbrt
```

This helps uncover:

- hidden subdomains
- forgotten services

10.6.2 dnsenum

`dnsenum` is another DNS enumeration tool.

```
dnsenumexample.com
```

It automates:

- DNS record discovery
- subdomain enumeration

10.6.3 nslookup (Windows & Linux)

Used for basic DNS queries.

```
nslookup example.com
```

Still useful for:

- quick checks
 - manual verification
-

10.7 TCP & UDP Reconnaissance

10.7.1 TCP Connect Scanning (Concept)

TCP scanning relies on the **three-way handshake**:

1. SYN
2. SYN-ACK
3. ACK

If handshake completes → port is **open**.

10.7.2 TCP Scan Using Netcat

```
nc -nvv -w1 -z192.168.50.1523388-3390
```

Options explained:

- **z** → zero I/O (scan mode)
 - **w** → timeout
 - **n** → no DNS resolution
-

10.7.3 UDP Scanning

UDP is **stateless**, so scanning works differently.

```
nc -nv -u -z -w1192.168.50.149120-123
```

No response often indicates:

- open or filtered port
-

10.8 Nmap Scanning & Traffic Footprint

10.8.1 Default Nmap Scan

By default, Nmap scans:

- top **1000 most common TCP ports**

Understanding scan footprint is important to:

- avoid detection
- respect scope

10.8.2 Measuring Scan Traffic Using iptables

Insert monitoring rules:

```
sudo iptables -I INPUT 1 -s 192.168.50.149 -j ACCEPT
sudo iptables -I OUTPUT 1 -d 192.168.50.149 -j ACCEPT
sudo iptables -Z
```

After scan:

```
sudo iptables -vn -L
```

This shows:

- packets sent
- bandwidth consumed

10.8.3 Full Port Scan

```
nmap -p1-65535 192.168.50.149
```

More coverage → more noise.

10.9 SMB Enumeration

10.9.1 What Is SMB Enumeration

SMB enumeration collects information from **Windows file-sharing services**.

SMB is used for:

- file sharing
 - authentication
 - printer services
-

10.9.2 Why SMB Enumeration Matters

Misconfigured SMB can leak:

- usernames
- shares
- OS details
- domain info

SMB enumeration enables:

- lateral movement
 - privilege escalation planning
-

10.9.3 Common SMB Enumeration Tools

Nmap

```
nmap -p139,445 --script smb-enum-shares,smb-enum-users <target>
```

```
nmap -v -p139,445--script smb-os-discovery <target>
```

nbtscan

```
sudo nbtscan -r192.168.50.0/24
```

Used for NetBIOS discovery.

smbclient

```
smbclient -L//<target-ip> -N
```

- **N** → null session.
-

enum4linux

```
enum4linux <target-ip>
```

Beginner-friendly SMB enumeration.

crackmapexec

```
crackmapexec smb <target-ip>
```

Used in real-world pentests.

smbmap

```
smbmap -H <target-ip>
```

Maps shares and permissions.

10.9.4 SMB Null Session

A **null session** allows access **without credentials**.

This is a **serious misconfiguration**.

10.10 SMTP Enumeration

10.10.1 What Is SMTP Enumeration

SMTP enumeration identifies **valid email users**.

It abuses server responses to confirm:

- existing users
 - naming conventions
-

10.10.2 SMTP Commands Used

```
VRFY username  
EXPN admin  
RCPTTO:user@example.com
```

10.10.3 SMTP Enumeration Tools

Nmap

```
nmap -p25 --script smtp-enum-users <target>
```

smtp-user-enum

```
smtp-user-enum -M VRFY -U users.txt -t <target>
```

Netcat

```
nc <target-ip> 25
```

Manual testing.

Metasploit

```
auxiliary/scanner/smtp/smtp_enum
```

10.11 SNMP Enumeration

10.11.1 What Is SNMP Enumeration

SNMP enumeration extracts information from:

- routers
 - switches
 - servers
 - printers
-

10.11.2 When SNMP Is Exposed

- UDP port **161 open**
 - weak/default community strings
-

10.11.3 SNMP Enumeration Tools

snmpwalk

```
snmpwalk -v2c -cpublic <target-ip>
```

snmp-check

```
snmp-check <target-ip>
```

nmap

```
nmap -sU -p161 --script snmp-info <target>
```

onesixtyone

```
onesixtyone <target-ip> -c community.txt
```

10.12 Where These Techniques Fit in VAPT Workflow

- Phase: **Reconnaissance / Enumeration**
- Comes after: basic scanning
- Comes before: exploitation
- Requires: strict scope adherence